

REACT

REACT REGELN (KOMPONENTE, HTML, JSX)

- Nur ein Rückgabewert: 1 parent element (Root Element) oder React.Fragment oder <>
- Naming der Komponente
 - UpperCamelCase
 - treffende Beschreibung der Komponente
 - Beispiele: LinkButton, InfoTooltip, DraftEditor
- HTML Attribute in JSX mit camelCase (ausser aria- und data-*)
 - `<div>` → `className`, `for` in Formularfeldern → `htmlFor`, `stroke-width` → `strokeWidth`
- HTML Elemente geschlossen, z.B. `<img />`, `<input />`
- Syntax wie in JS: Wert in {title}

REACT CODE

useReducer & UI Libraries

```
import { useReducer } from 'react';

type ReducerActionTypes = 'increment' | 'decrement';

const reducer = (
  state: { count: number }, // state ist Zahl von counter
  action: { type: ReducerActionTypes }, // + oder -
) => {
  switch (action.type) {
    case 'increment':
      console.log('incrementing');
      return { count: state.count + 1 };
    case 'decrement':
      console.log('decrementing');
      return { count: state.count - 1 };
    default:
      throw new Error('Unknown action type');
  }
};

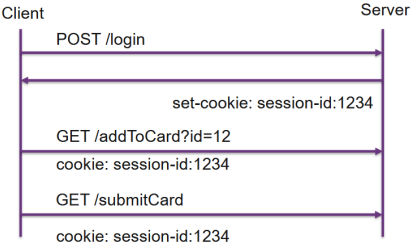
export const StaticCounter = () => {
  const actions: Record<type, ReducerActionTypes>[] = [
    { type: 'increment' },
    { type: 'increment' },
    { type: 'increment' },
    { type: 'decrement' },
  ];

  // static use of the reducer
  const finalState = actions.reduce(reducer, { count: 0 });
  return <static reducer result: {finalState.count}</>;
};

export const Counter = () => {
  // use reducer with useReducer hook
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <div>useReducer</div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
    </div>
  );
};
```

EXPRESSJS

COOKIES



Server: set-cookie:name=value;Expires=Wed, 09 Jun 2029 10:18:14 GMT Client:
cookie:name=value; name2=value2

Cookies für ExpressJS Session: app.use(session({ secret: '1234567', resave: false, saveUninitialized: true}));

```
const app = express();
app.use(cookieParser('secret'));
```

```
app.get('/cookieDemo/{*splat}', function (req, res) {
  console.log(JSON.stringify(req.cookies));
  console.log(JSON.stringify(req.signedCookies));
  res.cookie('url', req.url);
  res.cookie('signedUrl', req.url, {signed: true});
});
```

```
if (req.cookies.url) {
  res.end('dein letzter besuch: ' + req.cookies.url);
} else {
  res.end('dein erster besuch?!');
}
```

```
app.listen(3000, function () {
  console.log('listening on http://localhost:3000');
});
```

JSON WEB TOKEN (JWT) JSON-BASED OPEN STANDARD (RFC 7519).

HTTP-Header: Authorization: Bearer <token>

EXPRESS TESTAT CODE

index.js

see express.js demo

app.js

```
import express, { NextFunction, Request, Response } from 'express';
import path from 'path';
import cors from 'cors';
```

```
import { userRoutes } from './routes/user-routes';
import { accountRoutes } from './routes/account-routes';
import { transactionRoutes } from './routes/transaction-routes';
import { CONFIG } from './config';
import { expressJwt } from 'express-jwt';
import { HttpError } from './services/http-error';

export const app = express();

app.use(cors());
app.use(express.static(path.resolve('public')));
app.use(express.json());

app.use(
  expressJwt(CONFIG.jwt.validate).unless({
    path: [/\/login/, \/register/],
  }),
);

// app.use('/users', userRoutes);
// app.use('/accounts', accountRoutes);
app.use('/transactions', transactionRoutes);

app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof HttpError) {
    return res.status(err.status).json({ message: err.message });
  }
  if (err.name === 'UnauthorizedError') {
    return res.status(401).json({ message: 'No token / Invalid token provided' });
  }
  return res.status(400).json({ message: 'something went wrong', err });
});

routes.transactionRoutes.ts
import express from 'express';
import { transactionController } from '../controller/transaction-controller';

const router = express.Router();

router.post('/', transactionController.create);
// router.get('/', transactionController.getTransactions);
// router.get('/:accountNr', transactionController.getTransactions);

export const transactionRoutes = router;

controller/transaction-controller.ts
import { Request, Response } from 'express';
import { HttpError } from '../services/http-error';
import { SchemaUtil } from '../util/schema-util';
import { AddTransactionSchema, FindTransactionSchema, transactionService } from '../services/transaction-service';

export class TransactionController {
  public create = async (req: Request, res: Response) => {
    const auth = req.headers.authorization;
    if (!auth) { throw new HttpError(401, 'no token provided'); }
    if (!req.auth.userId) { throw new HttpError(401, 'Unauthorized'); }

    const data = SchemaUtil.parseOrThrow(AddTransactionSchema, req.body);

    const userUid = req.auth.userId;

    // import { accountService } from '../services/account-service';
    const isOwner = await accountService.isOwner(data.from, userUid);

    if (!isOwner) { throw new HttpError(402, 'Inconnect user'); }
    await transactionService.create(data);
    return res.status(200).json({ success: true });
  };
};
```

```
const userUid = req.auth.userId;

// import { accountService } from '../services/account-service';
const isOwner = await accountService.isOwner(data.from, userUid);

(accountService code at end)
```

```
if (!isOwner) { throw new HttpError(402, 'Inconnect user'); }
await transactionService.create(data);
return res.status(200).json({ success: true });
};
```

```
services/transaction-service.ts
import Database from '@seald-io/nedb';
import { z } from 'zod';
import { accountService } from '../account-service';
import { HttpError } from '../http-error';

const TransactionSchema = z.object({
  from: z.number(), // can be float
  amount: z.number().int(),
  date: z.date(), to: ..., total: ...
}); type Transaction = z.infer<typeof TransactionSchema>;

export const AddTransactionSchema = z.object({
  amount: z.number().int().positive(), from: ..., to: ...,
}); type AddTransaction = z.infer<typeof AddTransactionSchema>;

type FindTransactionResult = {
  docs: Transaction[]; count: number; skip: number; docCount: number;
};

export const FindTransactionSchema = z.object({
  accountNr: z.coerce.number(),
  count: z.coerce.number().int().positive().optional().default(10),
  skip: z.coerce.number().int().nonnegative().optional().default(0),
}); type FindTransaction = z.infer<typeof FindTransactionSchema>;

export class TransactionService {
  private db: Database<Transaction>;

  constructor() { this.db = new Database({}); }

  async create(data: AddTransaction): Promise<void> {
    const from = await accountService.get(data.from);
    if (from.balance < data.amount) {
      throw new HttpError(400, 'Insufficient funds');
    }
    // throw error if amount < 0, sending to yourself..

    const newFromBalance = from.balance - data.amount;
    const to = await accountService.get(data.to);
    const newToBalance = to.balance + data.amount;

    const date = new Date();
    await this.db.insertAsync({ from: data.from, ... });
    await accountService.update(from.accountNr, { balance: newFromBalance });
    await accountService.update(to.accountNr, { balance: newToBalance });
  } // transactionService.create

  async find(data: FindTransaction): Promise<FindTransactionResult> {
    const filter = {
      $or: [{ from: data.accountNr }, { to: data.accountNr }],
    };
  };
};
```

```
const limit = data.count < 5 ? data.count : 5;

let transactions: Transaction[] = await this.db
  .findAsync(filter)
  .sort({ date: -1 })
  .skip(data.skip,
    docCount: transactionCount,
    limit(limit);

const transactionCount = await this.db.countAsync(filter);

return {
  docs: transactions,
  count: transactions.length,
  skip: data.skip,
  docCount: transactionCount,
};
} // transactionService.find
} export const transactionService = new TransactionService();
```

services/account-service.ts

```
// imports and define schemas like transaction-service.ts
export class AccountService {
  // private db with constructor like transaction-service
  // here is the file version:
  constructor() {
    this.db = new Database({ filename: './data/account.db', autoload: true });
    // index und stellt Einzigartigkeit sicher:
    this.db.ensureIndexAsync({ fieldName: 'owner', unique: true });
    this.db.ensureIndexAsync({ fieldName: 'accountNr', unique: true });
  }

  // const isOwner = await accountService.isOwner(data.from, userUid);
  async isOwner(accountNr: number, userId: string) {
    const account = await this.getAccountNr();
    return account.owner === userId;
  }

  // const to = await accountService.get(data.to);
  async get(accountNr: number) {
    const account = await this.db.findOneAsync({ accountNr });
    if (!account) { throw new HttpError(404, `${accountNr} not Found`); }
    return account;
  }

  // await accountService.update(to.accountNr, { balance: newToBalance });
  async update(accountNr: number, data: Update) {
    const account = await this.db.updateAsync(
      { accountNr }, { $set: data }, { multi: false, returnUpdatedDocs: true },
    );
    if (!account.affectedDocuments) { throw new HttpError(404, `${accountNr} not Found`); }
    return true;
  }
};
```

```
// await accountService.update(to.accountNr, { balance: newToBalance });
async update(accountNr: number, data: Update) {
  const account = await this.db.updateAsync(
    { accountNr }, { $set: data }, { multi: false, returnUpdatedDocs: true },
  );
  if (!account.affectedDocuments) { throw new HttpError(404, `${accountNr} not Found`); }
  return true;
}
```

ACCESSIBILITY

Zielgruppen: Tastaturbenutzer, Screenreader (nicht UI sondern Acc.Tree), Sehschwächen, SEO+KI
Automatisiert prüfbar: fehlende Attribute (alt, label, lang), leere Links / Buttons (z.B. nur icon dann nutze aria-label="Zur Startseite"), Kontrastwerte (Farbkontrast), input mit placeholder statt label, z.B. mit Linter; Storybook Addon
manuell prüfen: schlechte UX / Interaktion, unklare Inhalte ("click here") sollten klar verständlich sein, schlechte Screen Reader Experience, Keyboard-Navigation im Kontext (Tabs), Button ist div statt <button type="button">..
Weitere typische Probleme: Alt-Text vorhanden → aber inhaltlich nutzlos ("image.jpg"), Label vorhanden → aber falsch verknüpft, Lighthouse Score hoch → trotzdem nicht nutzbar, Drag & Drop ohne Alternative
Accessibility wird oft falsch gemessen: Tools prüfen, was einfach messbar ist aber Nutzer erleben etwas anderes
die 3 goldenen Regeln: Semantik vor Styling (HTML native Elemente), Bedienbarkeit sicherstellen (Tastatur), Verständliche Benennung

Fixed Page

```
<main class="page">
<h1>Broken Demo</h1>
<nav class="card"> // semantic nav
<div>Navigation</div> // text for Link v
<a class="icon-link home-icon" href="#" index.html" aria-label="Zur
Startseite"></a>
</div>
<div class="card">
<h2 id="search-heading">Suche</h2>
<form class="search-row" role="search" aria-labelledby="search-heading">
<label for="search" class="visually-hidden">Suchbegriff</label>
<input type="search"> // has a label ^
<button class="icon-btn search-icon" type="button" aria-label="Suche
starten">
</button>
</form>
</div>
<div class="card">
<h2 id="form-heading">Formular</h2>
<form class="form" aria-labelledby="form-heading">
<label for="name">Name</label>
<input id="name" type="text">
-
<button class="submit-btn" onclick="submit()">Absenden</button>
</form> -
```

TESTING

Warum: Änderungen erzeugen neue Fehler (Regressionen), moderne Webprojekte bestehen aus vielen Teilen, AI generiert schneller neuen Code, manuelles Prüfen skaliert schlecht, automatisierte Checks geben schnelles Feedback.
Gute Tests prüfen Grenzfälle: 0, negativ, sehr gross, leer, null, falscher Typ und unerwartete Eingaben.
Test-Smells: Flaky Tests Abhängig von Zeit, Netzwerk, Reihenfolge, Implementation Detail Testing, z.B. jeder 2. funktioniert, übermäßiges Mocking, dann refactor, Architektur prüfen

Arten von Checks

tsconfig.json mit "compilerOptions": { "strict": true, ...: strengere Typprüfung, ungenutzte Variablen erkennen, Compiler-Regeln definieren
eslint.config.js mit rules: { "no-unused-vars": "error", "jsx-a11y/alt-text": "warn" ...: Coding Standards, Accessibility-Regeln, Qualitätsregeln für React
Static Checks (Qualität): erkennen potenzielle Probleme, prüfen Typen, prüfen Regeln, prüfen Accessibility, helfen Fehler vermeiden. Erkennen: falschen Typ, unbenutzter Code, fehlendes Label, fehlendes Accessibility-label, falsche HTML Struktur, ungültige Props. TypeScript Typen und schnittstellen, ESLint code-Regeln und problematische Patterns, Accessibility-Linting semantische Ur-Probleme
Prettier (Lesbarkeit): formatiert Code automatisch, keine Qualitätsprüfung, keine Typprüfung, keine fachlichen Regeln

Testebenen - bei React als Test Trophäe

Static Checks: prüft Struktur (TypeScript), soweit wie möglich, siehe oben
Unit Tests: prüft einzelne Funktionen (calculateFee()), viele machen, schnell, isoliert, falsche Berechnung, Business Logik
Integration Tests: prüft Zusammenspiel (API + Daten), (z.B. für Frontend mit Playwright, React Testing Lib, MSV), Unit Tests + falsche API Antwort, Login
E2E Tests: prüft User Flows (Login), mit Playwright/React Testing Lib, langsam, Integr. Tests + fehlendes Label wie Static checks

weitere Infos

API Tests: liegen zwischen Unit Tests und E2E Tests. Sie prüfen das Zusammenspiel von Routing, Request-Verarbeitung und Response-Format. nicht produktive UI, Frontend Darstellung, UX
(Unit Tests) Arrange-Act-Assert: Was war vorbereitet? Was wurde getan? Was ist herausgekommen?
Gut testbar = gute Architektur: kleine Funktionen, Funktionen ohne Nebeneffekte (pure), klare Schnittstellen, lose Kopplung, Dependency Injection (statt direkte Abhängigkeiten), Zustände nicht global

Test Coverage

Statement Coverage: Wurden alle Anweisungen ausgeführt?
Branch Coverage: Wurden alle if/else-Zweige getestet? 80% Branch wertvoller als 95% Lines.
Function Coverage: Wurden alle Funktionen aufgerufen?
Line Coverage: Wurden alle Codezeilen ausgeführt? 70–80%
weitere Zahlen: Kritische Pfade = 100% (Auth, Geld, Datenverlust) und UI = 50-60%, E2E für Rest.

```
Mocking, z.B. Test Doubles:
Mock: Wie Spy, aber mit Verhaltens-Erwartungen, vi.fn().mockReturnValue(42)
Spy: Echte Funktion + Aufrufe werden mitprotokolliert, vi.spyOn(obj, "method")
Dummy: Nur als Platzhalter - wird nie wirklich benutzt, z.B. null als Logger-Argument
Stub: Liefert vorgegebene Antworten, getCurrentHour = () => 19
Fake: Funktionierende, z.B. vereinfachte Implementierung In-Memory DB statt PostgreSQL

E2E Test mit Playwright
test("user can log in", async ({ page }) => {
  await page.goto("/");
  // getByLabel nutzt sichtbare Labels und fördert dadurch zugängliche Forms.
  await page.getByLabel("Password").fill("secret");
  // getByRole = semantischer und robuster als CSS oder generische Selektoren.
  await page.getByRole("button", { name: "Login" }).click();
  await expect(page.getByRole("status")).toHaveText("Login erfolgreich");
});

Unit + API Test mit Vitest und request von Supertest
describe("calc", () => { test("calculates ...", () => {
  const getCurrentHour = () => 18; // Arrange
  expect(calc(1000, getCurrentHour)).toBe(18); // Act and Assert
  expect(() => calc(0, false)).toThrow();
}); });

describe("accounts API", () => {
  test("GET /accounts/1 returns an account", async () => {
    const res = await request(app).get("/accounts/1");
    // request(app).post("/accounts/1/deposit").send({ amount: -50 });
    expect(res.status).toBe(200);
    expect(res.body).toMatchObject({ id: 1, owner: "Ada" }); // Object{error: ""}
    expect(typeof res.body.balance).toBe("number");
  });
});
```

SECURITY

XSS Schutz

1. **Output Escaping:** `htmlspecialchars({input})`, `{input}` , `htmlspecialchars({input})` // hbs,
2. **Sanitizing** (Zeichen entfernen) **beim Speichern** `sanitize({input})`
3. **Content Security Policy (CSP):** blockiert Script-Ausführung im Browser + Server sendet Info im Seiten-Header. Z.B. mit «Helmet» `app.use(helmet.contentSecurityPolicy({...})).` CSP = wohin der Browser (Fetch)-Requests senden darf.
4. **HttpOnlyCookies:** Start JWT im `localStorage / sessionStorage`

```
res.cookie('session', token, {
  httpOnly: true, // nicht via document.cookie lesbar
  secure: true, // nur über HTTPS
  sameSite: 'lax' // CSRF-Schutz
});
```

Angriffs-Szenario
Setze `user-input` auf: `<script>document.body.insertAdjacentHTML('afterbegin', '<p style="color:red">red*HACKED</p>')</script>`

CORS

CORS (Cross Origin Resource Sharing) : Browser entscheidet ob Response für JS zugänglich ist, *nur* `Access-Control-Allow-Origin`. Nicht verwechseln mit CSP: Content Security Policy (CSP) begrenzt, was der Browser laden und ausführen darf → reduziert Schaden bei XSS (aber ersetzt kein Escaping)

- Angriffs-Szenario**
1. User nutzt `app.example.com` mit Daten von `api.example.com`.
 2. Nutzer öffnet aus Versehen `evil.com` im Browser
 3. `evil.com` Seite versucht `request: fetch("https://api.example.com/user", {credentials: "include"})` // Cookies mitgesendet

CSRF CROSS-SITE REQUEST FORGERY

- Angriffs-Szenario**
1. Nutzer ist eingeloggt (Session aktiv)
 2. Nutzer besucht eine Fremde Seite (Attacker)
 3. Diese Seite sendet (Form-Submit oder fetch) eine Anfrage an unsere App
 4. Browser sendet automatisch das Session-Cookie mit
 5. Server führt die Aktion aus (denkt: legitimer Nutzer)

CSRF Schutz
CSRF Token Server prüft: kommt Request aus meiner App? und **Keine State Changes** via GET → besonders unsicher

```
// Formular (Client)
<form method="POST" action="/change-email">
  <input name="email" />
  <input type="hidden" name="csrfToken" value="{{token}}" />
  <button>Save</button>
</form>
// Server (Express Form-Handling)
app.post('/change-email', (req, res) => {
  if (req.body.csrfToken === session.csrfToken) {
    return res.status(403).send("Forbidden");
  }
  email = req.body.email;
  res.send("OK");
});
```

Cookies

ermöglichen Authentication nicht Authorisation Wichtige Cookie Optionen

- `httpOnly` → Cookie nicht via JS zugreifbar
- `secure` → nur über https gesendet
- `sameSite: lax` (default, gesendet bei `same-site + top-level Navigation/GET`); `None` (immer), `[strict]`
- `maxAge / expires`: Lebensdauer

LOGIN UND PASSWORT MANAGEMENT

- UX: Re-Authentication (Passwort bestätigen) und Bestätigungs-Dialog vor irreversiblen Aktionen *für Security und gutes Design*
- Login selber bauen fast immer schlechte Idee, nutze existierende Dienste (Auth0, Firebase Auth...) oder Passwortlos (Magic Link, Passkey).
- schlechte UX = Passwort-Reuse, Fehler → Frust → Workarounds.

Sign-up Form Best Practices
Nutze den Browser: richtige HTML-Elemente `<form>`, `<label>`, `<button>`, `<type>`, `required`, `autocomplete` nutzen

Reduziere «Friction»: keine doppelte Eingabe (Email / Passwort), klare Fehlermeldungen, Passwortregeln klar anzeigen

Password UX richtig machen: Browser Password Manager nutzen, `autocomplete="new-password"/<current-password>`, Show password anbieten

Mobile First & Accessibility: Grosse Inputs und Buttons, Jedes Feld mit `<label>`

IDOR

Angriffs-Szenario
Der Angreifer loggt sich ein und manipuliert die URL der StockPortfolio Seite von `www.retireEasy.com/stockportfolio/attackerUID` zu `www.retireEasy.com/stockportfolio/victimID`. Diese Seite wird angezeigt.

IDOR Schutz

```
app.get('/api/documents/:id', async (req, res) => {
  const doc = await db.getDocument(req.params.id);
  if (!doc || doc.ownerId !== req.session.userId) {
    return res.status(404).json({}); // Enumeration verhindern
  }
  res.json(doc);
});
```

Object-level Authorization-Check: Gehört das Objekt dem aktuellen User?

OWASP TOP 10

A01-2025: Broken Access Control (IDOR, CSRF) A03-2025: Software Supply Chain Failures (Vulnerabel & Outdated Components → NPM) A05-2025: Injection: Cross-Site-Scripting (XSS), JS Code Injection, SQL Injection A06-2025: Insecure Design (→ Meine Impfungen: Nicht gesetzeskonforme und überwindbare Legitimationsprüfung (Brute-Force + IDOR + CSRF + XSS))

OWASP Top 10:	
Cross-Site-Scripting	Replay Attack
Remote Code Execution	Cryptographic Failures
Insecure Direct Object References	Identification & Authentication Failure
Cross-Site Request Forgery	

Cross Site Scripting (XSS): Website besitzt eine XSS-Verwundbarkeit, wenn es möglich ist den Server so zu manipulieren, dass Schadcode (JavaScript) eines Angreifers an Nutzer ausgeliefert wird und im Browser dieser Nutzer ausgeführt wird. Gegenmassnahme mit Input Sanitation.

Remote Code Execution: Webservers besitzt eine Code Injection "Vulnerability" wenn ein Angreifer den Server dazu bringen kann vom Angreifer eingeschleusten Code zum Ausführen zu bringen.

Broken Access Control: Beim behandeln von Formular-Submission-Requests (GET und POST) sollte überprüft werden, dass dies von einem vom Server für den Nutzer «Frisch» ausgelieferte Formulare stammt.

Cryptographic Failures: Passwort oder Token wird nicht verschlüsselt übertragen. Unabhängig davon ob Query-Body/Request-Parameter.

Identification & Authentication Failure: Bei Problemen bei der Authentisierung und dem Session Management können externe Angreifer oder Angreifer mit einem validen Login auf Informationen zugreifen, welche nicht für sie bestimmt sind

Sign-up Form Best Practices:

- Use meaningful HTML elements: form, input, label and button
- Label each input with a label
- Use element attributes to access built-in browser features: type, name, autocomplete, required.
- Use `autocomplete="new-password"` and `id="new-password"` for the password input in a sign-up form, and for the new password in a reset-password form.
- Provide Show password functionality.
- Don't double-up inputs. Don't force users to enter emails or passwords twice.

Content Security Policy (CSP): CSPs ermöglichen die Ausführung von böartigem Code auf Webseiten zu verhindern. Quellen von Ressourcen (Skripten, Bilder etc.) können eingeschränkt werden. CSPs werden im HTTP Header definiert

Cross-Origin Resource Sharing (CORS): CORS Header ermöglichen Ressourcen (wie z.B. Bilder, Skripte oder Daten) von einem anderen Ursprungsort als der eigenen "Origin" zu laden.